Universidade Federal do Rio Grande do Sul
Instituto de Informática

Disciplina: INFO1120 - Desenvolvimento de Software
Professor: Marcelo Soares Pimenta

Estudo de Bad Smells e Refatoração

André Schaidhauer Luckmann 00601117
Arthur Von Groll dos Santos 00602432
Carlos Eduardo Boesel Morari 00609369
Giovani Ferreira de Alencoi 00601395

Porto Alegre
6 de outubro de 2026

1 Introdução

Um *bad smell* indica que a estrutura de um programa pode dificultar sua compreensão, manutenção e evolução. Ele não representa, por si só, um defeito funcional, mas revela problemas na estrutura do código. Antes de propor mudanças ou funcionalidades, é necessário analisar o código e seu contexto, pois estruturas inadequadas podem esconder fluxos, regras de negócio e decisões arquiteturais antigas.

+Para tratar *bad smells*, aplica-se a refatoração, que melhora a manutenibilidade sem alterar o comportamento observável do programa.

2 Seleção dos smells

Selecionamos três smells do texto-base (SMELL-1): Feature Envy, Objetos Mutáveis e Comentários. Os dois smells externos (SMELL-2) são Middle Man e Shotgun Surgery.

3 Descrição dos bad smells

3.1 Feature Envy (SMELL-1)

O bad smell *Feature Envy* trata de quando temos uma classe que tem métodos muito pouco ligados (baixa coesão) com os atributos e métodos da própria classe, e mais interessados nos métodos/atributos de outra classe. Esse *bad smell* pode aparecer em um código como o exemplo abaixo:

```
1  public class Sphere {
2      private int radius, x, y;
3      private float surfaceArea;
4      private float volume;
5
6      public Sphere(int radius, int x, int y) {
7          this.radius = radius;
8          this.x = x;
9          this.y = y;
10         this.surfaceArea = this.calculateSurfaceArea();
11         this.volume = this.calculateVolume();
12     }
13
14     public int getX() {
15         return this.x;
16     }
17
18     public int getY() {
19         return this.y;
20     }
21
22     public int getRadius() {
23         return this.radius;
24     }
25
26     public float calculateSurfaceArea()
```

```
    {
        //...
    }

    public float calculateVolume()
    {
        //...
    }
}

public class Drawer {
    public void drawSphereWithShadow(GraphicsEngine engine, Sphere sphere) {

        int startX = sphere.getX() - sphere.getRadius();
        int startY = sphere.getY() - sphere.getRadius();
        int size = sphere.getRadius() * 2;

        int shadowX = startX + 5;
        int shadowY = startY + 10;

        engine.drawOval(shadowX, shadowY, size, size, GraphicsEngine.COLOR_GREY)
            ;
        engine.drawOval(startX, startY, size, size, GraphicsEngine.COLOR_BLUE);
    }
}
```

Listing 1: Feature Envy: antes da refatoração

No exemplo acima, temos uma simples classe “Sphere”, que se trata de uma esfera com raio, volume e área da superfície. Logo abaixo, temos uma classe “Drawer”, que é uma classe responsável por desenhar uma dada esfera na tela para que o usuário possa visualizá-la.

De início, percebe-se que o método usado para desenhar a esfera é o “drawSphereWithShadow”, dentro de “Drawer”. No entanto, esse método não usa nenhum atributo da classe Drawer (que, inclusive, não apresenta nenhum atributo), e faz uso de vários atributos da classe “Sphere”. Isso indica, claramente, um erro de design do código: A classe “Drawer” não tem atributos e é apenas um conjunto de métodos que tratam mais sobre as classes “GraphicsEngine” e “Sphere”. Seus comportamentos poderiam ser realocados para lugares onde fariam mais sentido com um comportamento muito mais coeso, o que facilitaria manutenções futuras e traria uma melhor legibilidade do código (não seria necessário acessar várias classes para entender o comportamento). Um exemplo de refatoração que causa traz esses benefícios é o seguinte:

```
public interface Drawable {
    public void draw(GraphicsEngine engine);
}

public class Sphere implements Drawable {
    private int radius, x, y;
    private float surfaceArea;
    private float volume;

```

```
10      public Sphere(int radius, int x, int y) {
11          this.radius = radius;
12          this.x = x;
13          this.y = y;
14          this.surfaceArea = this.calculateSurfaceArea();
15          this.volume = this.calculateVolume();
16      }
17
18      public int getX() {
19          return this.x;
20      }
21
22      public int getY() {
23          return this.y;
24      }
25
26      public int getRadius() {
27          return this.radius;
28      }
29
30      public float calculateSurfaceArea()
31      {
32          //...
33      }
34
35      public float calculateVolume()
36      {
37          //...
38      }
39
40      @Override
41      public void draw(GraphicsEngine engine) {
42          int startX = this.getX() - this.getRadius();
43          int startY = this.getY() - this.getRadius();
44          int size = this.getRadius() * 2;
45
46          int shadowX = startX + 5;
47          int shadowY = startY + 10;
48
49          engine.drawOval(shadowX, shadowY, size, size, GraphicsEngine.COLOR_GREY)
                ;
50          engine.drawOval(startX, startY, size, size, GraphicsEngine.COLOR_BLUE);
51      }
52  }
```

Listing 2: Feature Envy: comportamento movido para Sphere

Nesse caso, foi criada uma interface “Drawable”, que de certa forma, toma o lugar da classe “Drawer”, mas permite passar a responsabilidade para dentro da classe “Sphere”. Isso permite que “Sphere” tenha uma coesão muito maior (ela tem acesso a todas as informações da própria esfera e tudo que é necessário para desenhá-la). Isso facilita a questão de legibilidade (tudo necessário para entender o que o método faz está na própria

classe, não é necessário vagar por múltiplos arquivos diferentes) e a manutenção pode ser toda realizada ali (escopo de modificação pequeno). Além disso, essa refatoração também aplica o princípio "Tell, Don't Ask": a classe externa não precisa ficar 'perguntando' pelos dados da esfera para tomar decisões, nós simplesmente "dizemos" à esfera para se desenhar, já que ela é a detentora de todas as informações necessárias para isso. Outra questão que é interessante de analizar nesse caso é que o método draw fica acoplado à **GraphicsEngine**, que poderia ser eliminado se invertessemos a dependência e fizessemos com que **GraphicsEngine** recebesse um objeto desenhável com todas as instruções de como se desenha o objeto, e a engine somente iria desenhar na tela os comandos recebidos. Porém, como a didática deste exemplo foca na resolução do smell *Feature Envy*, resolvemos manter essa implementação, mas estamos cientes das alternativas.

3.2 Objetos Mutáveis (SMELL-1)

O bad smell *Objetos Mutáveis* ocorre quando um objeto permite ser modificado após ser criado. O principal problema dessa prática é que compartilhar o objeto entre métodos gera o risco de ele ser modificado inesperadamente no fluxo do código.

Uma forma de reduzir esse risco é criar objetos imutáveis, cujo estado não muda após a construção. Em Java, a palavra reservada `final` impede a reatribuição de um objeto ou campo, mas não torna um objeto imutável por si só. Para isso, a classe não deve expor métodos que alterem seu estado; seus campos devem ser privados e finais; e referências a objetos mutáveis devem ser copiadas defensivamente ou substituídas por tipos imutáveis. Quando for necessário representar uma alteração, cria-se uma nova instância com o estado desejado. Veja o seguinte exemplo:

```
1  public class Agendamento {
2    private LocalDate data;
3
4    public Agendamento(final LocalDate data) {
5      this.data = data;
6    }
7
8    public void adiar(final long dias) {
9      this.data = this.data.addDays(dias);
10   }
11
12   public LocalDate getData() {
13     return this.data;
14   }
15 }
```

Listing 3: Antes: objeto mutável compartilhado

Se o mesmo `Agendamento` for compartilhado por outros objetos, uma chamada a `adiar()` altera o estado observado por todos os seus clientes, o que pode ser uma alteração indesejada no fluxo do código. Abaixo, temos um exemplo que preserva o objeto original e devolve uma nova versão.

```
1  public final class Agendamento {
2    private final LocalDate data;
3
4    public Agendamento(final LocalDate data) {
```

```
    this.data = data;
  }

  public Agendamento adiado(final long dias) {
    return new Agendamento(this.data.plusDays(dias));
  }

  public LocalDate getData() {
    return this.data;
  }
}
```

Listing 4: Depois: objeto imutável

Neste caso, `LocalDate` também é imutável. Se o campo fosse uma referência mutável, como `java.util.Date` ou uma lista, a classe precisaria copiar o valor no construtor e no método de acesso para não expor seu estado interno. Em resumo, ao usar objetos mutáveis, deve-se analizar se eles realmente foram projetados para serem mutáveis, e que sua mutabilidade é controlada adequadamente pelo programa. Caso exista a possibilidade de algum objetos ser alterado indevidamente, deve-se utilizar recursos para torná-lo imutável e eliminar imprevisibilidades no software.

3.3 Comentários (SMELL-1)

Comentários são um ótimo recurso para explicar o código para seres humano, porém, podem se tornar *bad smells* quando utilizados para compensar um código pouco expressivo. É interessante que o programador reflita sobre como tornar o trecho de código mais legível para que ele seja auto documentável. Dessa forma, os comentários presentes no código determinam nomenclaturas ou informações que o código por si só não explica diretamente. E poupa a escrita de linhas e comentários que o próprio código poderia explicar.

```
x = 6 # nota da primeira prova
y = 8 # nota da segunda prova
z = 9 # nota da terceira prova
m = (x + y + z) / 3 # media das notas do semestre
```

Listing 5: Comentário que encobre código pouco expressivo

Neste trecho de código não é possível identificar o que está sendo calculado propriamente pelo código em si, por conta da falta de nomes significativos. Para suprir essa deficiência o programador inseriu comentários para explicar o fluxo, gerando um *comment smell*, dado que os comentários não se fariam necessários se os nomes escolhidos para as variáveis fossem mais expressivos. Lembrando que comentários são muito importantes para registrar alguma regra de negócio, intenção ou motivo de decisão que o código não explica por si só, porém sempre que for possível melhorar a legibilidade do código de forma que evite comentários redundantes, melhor. Já dizia o Uncle Bob que código limpo deveria soar que nem uma boa prosa escrita: seguir uma lógica estruturada que conta uma história.

```
nota_prova_1 = 6
nota_prova_2 = 8
nota_prova_3 = 9
media_semestral = (nota_prova_1 + nota_prova_2 + nota_prova_3) / 3
```

Listing 6: Nomes que tornam o comentário desnecessário

A correção de *comment smell* pode envolver múltiplas técnicas de refactoring, dado que um *comment smell* pode derivar de diversos tipos de erros de programação. No exemplo acima, aplicar nomes significativos às variáveis foi suficiente. Mas poderia requerer diferentes técnicas de Refactoring, como mudança de estruturas ou até do fluxo do código que pode ser simplificado.

3.4 Middle Man

Middle Man identifica classes ou métodos que apenas repassam uma chamada a outro objeto. A refatoração *Remove Middle Man* permite que o cliente acesse o delegado quando a camada não acrescenta comportamento, validação ou proteção [2, 3].

No TTM, o grupo encontrou métodos de `Player` que delegavam operações para `AudioEngine`. A Listagem 7 mostra o caso observado.

```
public final class Player {
  private final Interpreter interpreter;
  private final AudioEngine audioEngine;

  public void play(final Music music) {
    final String text = this.interpreter.interpret(music.getContent());
    this.audioEngine.play(text);
  }
  public void pause() { this.audioEngine.pause(); }
  public void resume() { this.audioEngine.resume(); }
  public void stop() { this.audioEngine.stop(); }
  public void seek(final int index) {
    this.audioEngine.seek(index);
  }
}
```

Listing 7: Delegação em Player

O grupo considerou expor `audioEngine` diretamente, mas decidiu manter a abstração. `JFuguePlayer` também delega operações a `ManagedPlayer`; nessa posição, a camada protege o sistema de dependências diretas com a biblioteca JFugue e funciona como adaptador.

3.5 Shotgun Surgery

Shotgun Surgery ocorre quando uma mudança conceitualmente única exige pequenas alterações em várias classes [4]. O grupo usa o exemplo de uma regra de acesso premium, inicialmente duplicada em dois componentes.

4 Refatoração de Shotgun Surgery

```
public class ContentService {
  public void streamMovie(final User user) {
```

```
    if (!user.isActive()
        || !"PREMIUM".equals(user.getSubscriptionTier())) {
      throw new SecurityException("Acesso negado.");
    }
  }
}

public class DownloadController {
  public ResponseEntity<String> downloadFile(final User user) {
    if (!user.isActive()
        || !"PREMIUM".equals(user.getSubscriptionTier())) {
      return ResponseEntity.status(403).body("Apenas para usuarios Premium.");
    }
    return ResponseEntity.ok("Iniciando download...");
  }
}
```

Listing 8: Regra de acesso espalhada

Se o sistema aceitar um plano adicional, cada cópia da condição precisará mudar. A Listagem 9 concentra a regra em `User`.

```
public class User {
  private boolean active;
  private String subscriptionTier;

  public boolean hasPremiumAccess() {
    return this.active
        && ("PREMIUM".equals(this.subscriptionTier)
            || "VIP".equals(this.subscriptionTier));
  }
}

public class ContentService {
  public void streamMovie(final User user) {
    if (!user.hasPremiumAccess()) {
      throw new SecurityException("Acesso negado.");
    }
  }
}

public class DownloadController {
  public ResponseEntity<String> downloadFile(final User user) {
    if (!user.hasPremiumAccess()) {
      return ResponseEntity.status(403).body("Apenas para usuarios Premium.");
    }
    return ResponseEntity.ok("Iniciando download...");
  }
}
```

Listing 9: Regra de acesso concentrada em User

5 Discussão

O grupo escolheu os smells para cobrir problemas de coesão, estado, legibilidade, delegação e evolução. O exemplo de Shotgun Surgery reduz a duplicação e estabelece um ponto único para testar a política de acesso. O caso de Middle Man no TTM exigiu outra decisão: a equipe manteve a delegação porque ela preserva a separação entre a aplicação e a biblioteca de áudio.

6 Conclusão

Bad smells orientam a análise de código, mas não substituem a avaliação do contexto. Feature Envy, objetos mutáveis e comentários mostram problemas locais de projeto. Middle Man pode ser uma camada sem propósito ou uma fronteira necessária. Shotgun Surgery mostra o custo de espalhar uma regra por várias classes.

Referências

[1] VALENTE, Marco Tulio. *Engenharia de Software Moderna*. Capítulo 9: Refactoring. Disponível em: https://engsoftmoderna.info/cap9.html. Acesso em: 4 out. 2026.

[2] FOWLER, Martin. *Refactoring: Improving the Design of Existing Code*. 2. ed. Boston: Addison-Wesley, 2018.

[3] REFACTORING.GURU. *Middle Man*. Disponível em: https://refactoring.guru/smells/middle-man. Acesso em: 4 out. 2026.

[4] REFACTORING.GURU. *Shotgun Surgery*. Disponível em: https://refactoring.guru/smells/shotgun-surgery. Acesso em: 4 out. 2026.